AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

i-CREDIT HOURS ENGINEERING PROGRAMS

Loan Prediction Project - CSE387

Submitted to: Dr. Mahmoud Mounir & Eng. Esraa Karam

Submitted by Team 20:

1. AbdelRahman Adel 23P0144
2. Jana Hussien 2200090
3. Omar Ahmed 23P0100
4. Mariam Ibrahim 23P0123
5. Hla Waleed 23P0210
6. Malak Khaled 2300447
7. Kenzy Tarek 2301141

Contents

Abstract....4
1. Introduction....5
2. Dataset Description....6
3. Data Preprocessing (Phase 1)....6
3.1 Dropping Irrelevant Columns....6
3.2 Exploring Data Types & Relationships....6
4. Handling Missing Values....9
4.1 Gender....9
4.2 Married....9
4.3 Dependents....9
4.4 Self_Employed, Loan_Amount_Term, Credit_History....9
4.5 Saving Modes & Rules....9
5. Predicting Missing LoanAmount Using Regression....10
6. Feature Engineering....10
6.1 TotalIncome....10
6.2 Outlier Handling (Log + Capping)....11
7. Encoding Categorical Variables....12
8. Feature Scaling....12
9. Class Imbalance Handling....13
10. Modeling Approaches (Phase 2)....13
10.1 Decision Tree Classifier....14
10.2 KNN Classifier....15
10.3 Naïve Bayes Models....18
10.4 XGBoost Classifier....20
11. Model Comparison & Final Selection....24
12. New Customer Prediction Pipeline....25
13. Business Validation: Married + Semiurban Analysis....26
14. Conclusion....27
15. Appendix — GitHub Repository....27

1 Correlation Matrix of Numerical Columns 7
2 Relationship between Education and Dependents. 7
3 Relationship between Education and Married. 7
4 Relationship between Married and Dependants 8
5 Relationship between Education and Self_Employed. 8
6 Box Plots of LoanAmount and TotalIncome 11
7 Box Plots of LoanAmount (Log Transformed) and TotalIncome (Log Transformed). 11
8 Distribution of numerical before scaling. The plots illustrate the initial data distribution and highlight skewness or variability in the raw/log-transformed values. 12
9 Distribution of numerical features after scaling. The scaled plots demonstrate the effect of normalization/standardization, where features are now aligned to similar ranges and centered distributions, ... 12
10 Decision Tree Classifier 14
11 Decision Tree with Class Weights 14
12 Accuracy vs K Graph for KNN 15
13 KNN Pure Model 15
14 STRATEGY 2: KNN with Distance and Class Weights 16
15 STRATEGY 1: KNN with SMOTE 16
16 STRATEGY 3: KNN with ADASYN 16
17 STRATEGY 4: KNN with SMOTE-Tomek 16
18 STRATEGY 6: KNN with Smaller K + SMOTE 17
19 STRATEGY 5: KNN with Threshold Tuning 17
20 Naïve Bayes Threshold Accuracy Curve 18
21 Multinomial NB 19
22 Gaussian NB 19
23 Bernoulli NB 19
24 Gaussian NB + SMOTE 20
25 Gaussian NB + Class Weights 20
26 XGBoost Pure Model 21
27 STRATEGY 2: Using Class Weights 22
28 STRATEGY 1: SMOTE - Synthetic Minority Oversampling 22
29 STRATEGY 3: Threshold Tuning 22
30 STRATEGY 5: SMOTE + Tomek Links (Hybrid Approach) 23
31 STRATEGY 4: ADASYN - Adaptive Synthetic Sampling 23
32 New Customer Final Prediction Table 25
33 Loan Outcomes for Married Semiurban Customers 26
34 Loan Approval Distribution (Married + Semiurban) 26

Abstract

This project explores machine learning techniques for predicting loan approval outcomes using a structured, real-world banking dataset. The goal is to design a complete data mining pipeline that includes preprocessing, outlier handling, feature engineering, encoding, scaling, class imbalance strategies, model training, evaluation, and prediction for new customer entries.

A key principle guiding this work is that **banks do not make simple Yes/No decisions**. They evaluate applicants based on income stability, dependents, employment status, credit history, marital status, and location—factors deeply reflected in our modeling choices and our emphasis on interpretable models.

After extensive experimentation with XGBoost, Decision Tree, KNN, and Naïve Bayes—along with imbalance-handling strategies such as SMOTE, ADASYN, class weighting, and threshold tuning—the final selected model was the **Decision Tree with balanced class weights**, due to its performance and interpretability.

1. Introduction

Loan approval prediction is a classic classification problem used by financial institutions to assess risk and minimize losses. With the increasing number of loan applicants, manual evaluation becomes inefficient. Machine learning models can provide faster, more consistent, and more accurate predictions when trained properly on real-world data.

This report contains:

- Comprehensive preprocessing (Phase 1)
- Multiple model experiments (Phase 2)
- Full loan prediction pipeline for new applicants
- Real-world validation using demographic subgroups
- Figure placeholders for clear visualization

2. Dataset Description

The dataset (614 entries, 13 features) contains applicant financial and demographic attributes, including:

- Gender, Married, Dependents
- Education, Self_Employed
- ApplicantIncome, CoapplicantIncome
- LoanAmount, Loan_Amount_Term
- Credit_History
- Property_Area
- Loan_Status (target variable)

3. Data Preprocessing (Phase 1)

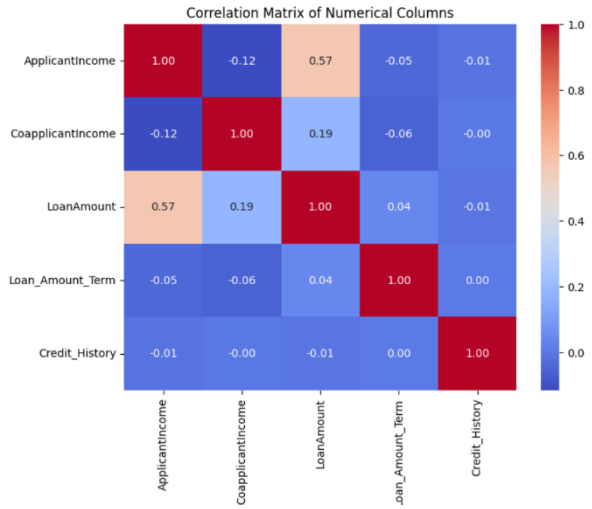
3.1 Dropping Irrelevant Columns

- Removed **Loan_ID** because it carries no predictive information.

3.2 Exploring Data Types & Relationships

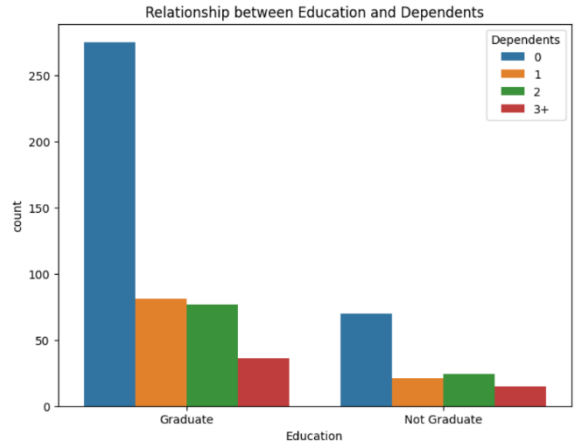
We inspected:

- Data types
- Unique values
- Missing values
- Relationships via correlation & countplots

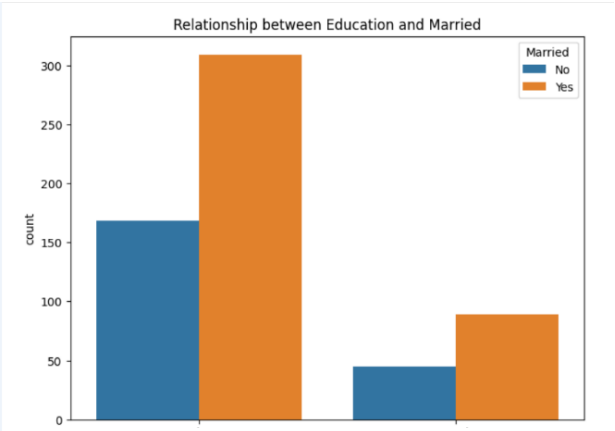


1 Correlation Matrix of Numerical Columns

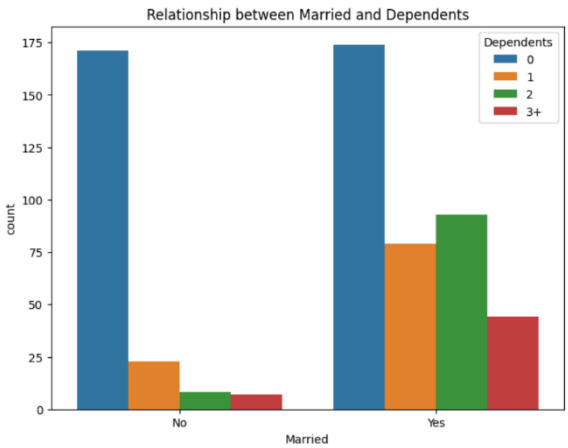
This heatmap (Phase 1) shows that ApplicantIncome correlates moderately with LoanAmount (~0.57).



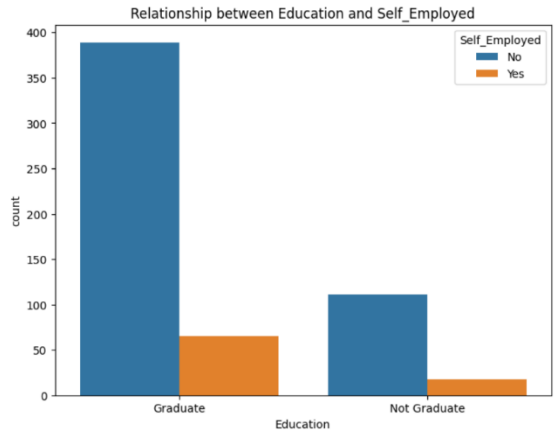
2 Relationship between Education and Dependents.



3 Relationship between Education and Married.



4 Relationship between Married and Dependants



5 Relationship between Education and Self_Employed.

These figures highlight meaningful demographic patterns important for imputation and modeling.

4. Handling Missing Values

4.1 Gender

Filled using mode.

4.2 Married

Applied a logical rule:

- Graduate → likely Married
- Not Graduate → likely Not Married

4.3 Dependents

Filled using mode **within the Married group**.

4.4 Self_Employed, Loan_Amount_Term, Credit_History

Filled using mode.

4.5 Saving Modes & Rules

Saved imputers using joblib to ensure **consistent preprocessing** for new customers.

5. Predicting Missing LoanAmount Using Regression

Instead of filling LoanAmount with mean/median, we used a **RandomForestRegressor** to predict missing values.

Steps:

1. Split data into missing vs. non-missing groups
2. Select correlated predictors
3. Apply OneHotEncoding to categorical features
4. Train RandomForestRegressor
5. Predict missing LoanAmount
6. Insert predictions into dataset

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
0	Male	No	0	Graduate	No	5849	0.0	144.07	360.0	1.0	Urban	Y
35	Male	Yes	0	Graduate	No	2275	2067.0	99.13	360.0	1.0	Urban	Y
63	Male	Yes	1	Graduate	No	4945	0.0	110.53	360.0	0.0	Rural	N
81	Male	Yes	1	Graduate	Yes	2395	0.0	84.55	360.0	1.0	Semiurban	Y
95	Male	No	0	Graduate	No	6782	0.0	140.76	360.0	1.0	Urban	N

This improves accuracy and reduces bias in the dataset.

6. Feature Engineering

6.1 TotalIncome

A highly important new feature:

TotalIncome = ApplicantIncome + CoapplicantIncome

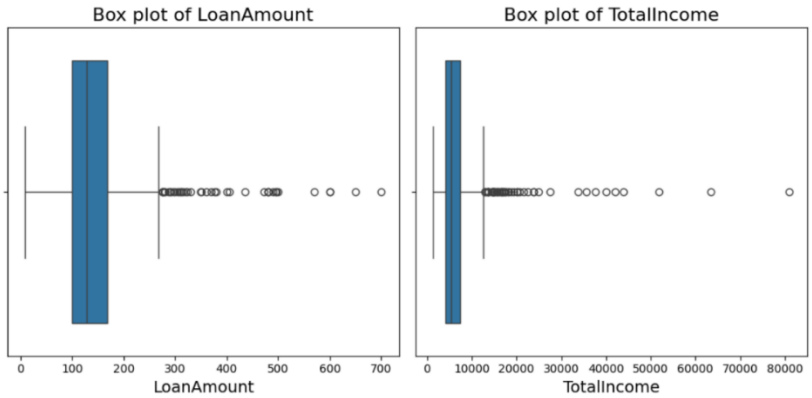
6.2 Outlier Handling (Log + Capping)

We avoided dropping outliers because:

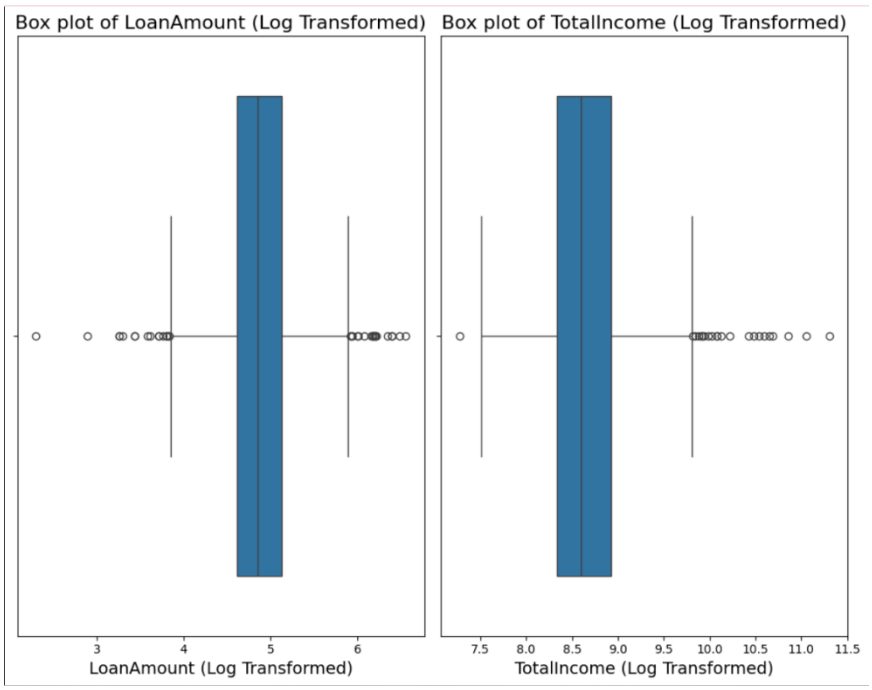
In banking, extreme values often represent real financial conditions. Removing them = data leakage.

Steps used:

1. Apply log transform to LoanAmount & TotalIncome
2. Apply gentle capping at the 99.5th percentile



6 Box Plots of LoanAmount and TotalIncome



7 Box Plots of LoanAmount (Log Transformed) and TotalIncome (Log Transformed).

These figures show improvement in distribution symmetry.

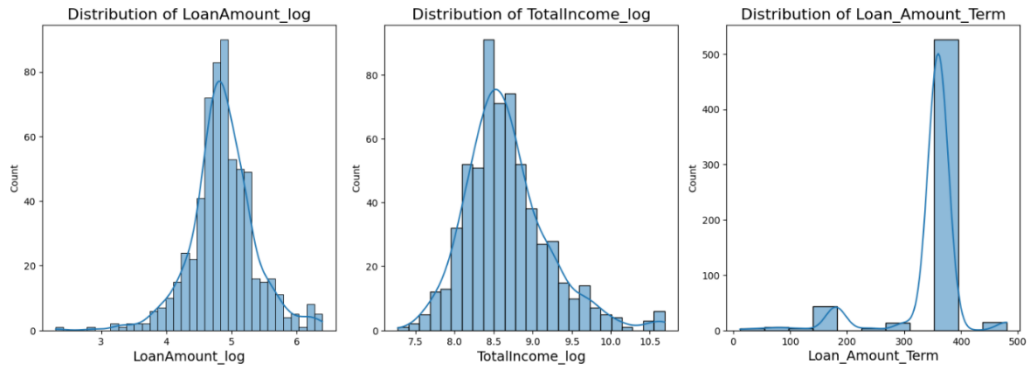
7. Encoding Categorical Variables

- **Label Encoding** → Dependents, Education
- **One-Hot Encoding** → Gender, Married, Self_Employed, Property_Area
- **Label Encoding Target** → Loan_Status (Y=1, N=0)

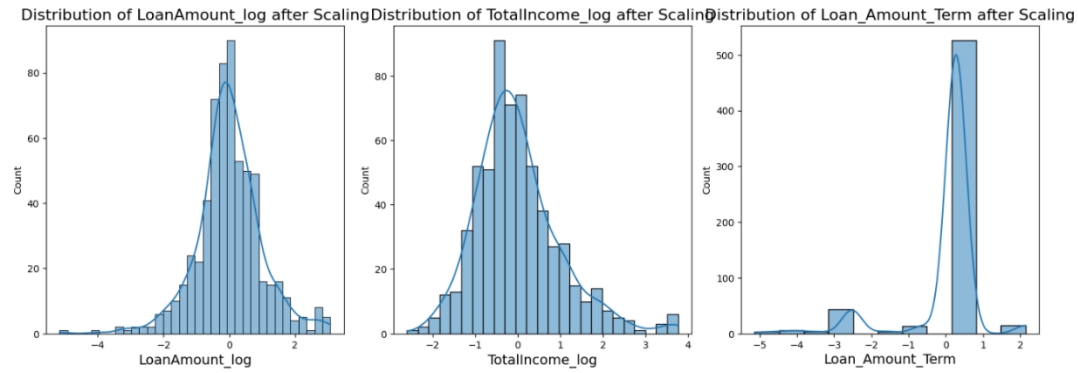
8. Feature Scaling

Scaled:

- LoanAmount_log
- TotalIncome_log
- Loan_Amount_Term



8 Distribution of numerical before scaling. The plots illustrate the initial data distribution and highlight skewness or variability in the raw/log-transformed values.



9 Distribution of numerical features after scaling. The scaled plots demonstrate the effect of normalization/standardization, where features are now aligned to similar ranges and centered distributions,

Scaling was crucial for KNN performance.

9. Class Imbalance Handling

Loan_Status was imbalanced:

- Approvals (1) ≈ double rejections (0)

This causes the model to favor approvals unless corrected.

Your team addressed it differently for each classifier:

- SMOTE
- ADASYN
- SMOTE–Tomek
- Class weights
- Threshold tuning

10. Modeling Approaches (Phase 2)

Before evaluating the models, it is important to highlight that our dataset suffered from a significant class imbalance: the number of approved loans ("Yes") was more than double the number of rejected loans ("No"). This imbalance can cause models to become biased toward predicting approvals. Therefore, during the modeling phase, we applied multiple imbalance-handling strategies—such as SMOTE, ADASYN, class weighting, and threshold tuning—to ensure fair and accurate predictions.
The models we focused on (Decision Tree, KNN, and Naïve Bayes) were the ones covered in our course, and we additionally included XGBoost as an advanced model to compare performance and validate our conclusions.

10.1 Decision Tree Classifier

Why DT performed best:

- Naturally interpretable
- Handles nonlinear patterns
- Works well with categorical & engineered features
- Balanced class weights reduce bias

```
Accuracy on test set: 0.8536585365853658
[[23 15]
 [ 3 82]]

Classification Report:

              precision    recall  f1-score   support

           0       0.88      0.61      0.72        38
           1       0.85      0.96      0.90        85

    accuracy                           0.85       123
   macro avg       0.86      0.78      0.81       123
weighted avg       0.86      0.85      0.84       123
```

10 Decision Tree Classifier

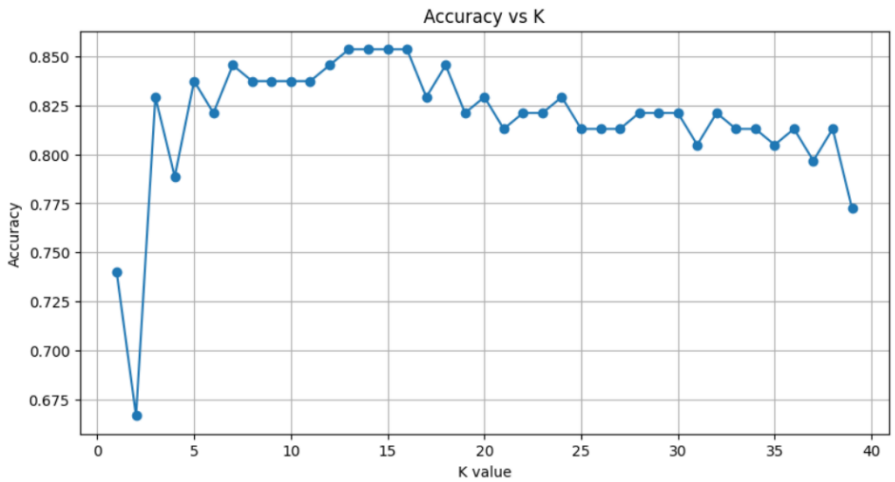
```
Decision Tree with Balanced Weights Results:
Accuracy: 0.8211
ROC-AUC: 0.7687

Confusion Matrix:
[[24 14]
 [ 8 77]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.63      0.69        38
           1       0.85      0.91      0.88        85

    accuracy                           0.82       123
   macro avg       0.80      0.77      0.78       123
weighted avg       0.82      0.82      0.82       123
```

11 Decision Tree with Class Weights

This is the **final chosen model**.

10.2 KNN Classifier

KNN requires scaling, so StandardScaler was used.



12 Accuracy vs K Graph for KNN

Grid Search optimized:

- n_neighbors
- metric
- weights

Additional tests:

- SMOTE
- ADASYN
- SMOTE-Tomek
- Threshold tuning

```
Final Test Accuracy: 0.8455284552845529
[[20 18]
 [ 1 84]]

Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.53      0.68        38
           1       0.82      0.99      0.90        85

    accuracy                           0.85       123
   macro avg       0.89      0.76      0.79       123
weighted avg       0.86      0.85      0.83       123
```

13 KNN Pure Model

```
STRATEGY 1: KNN with SMOTE

After SMOTE - class distribution:
Loan_Status
1    337
0    337
Name: count, dtype: int64

Best parameters: {'metric': 'manhattan', 'n_neighbors': 3}

KNN with SMOTE Results:
Accuracy: 0.7480
ROC-AUC: 0.7303

Confusion Matrix:
[[26 12]
 [19 66]]

Classification Report:
              precision    recall  f1-score   support

           0       0.58      0.68      0.63        38
           1       0.85      0.78      0.81        85

    accuracy                           0.75       123
   macro avg       0.71      0.73      0.72       123
weighted avg       0.76      0.75      0.75       123
```

15 STRATEGY 1: KNN with SMOTE

```
STRATEGY 2: KNN with Distance and Class Weights

KNN with Distance Weighting Results:
Accuracy: 0.8374
ROC-AUC: 0.7587

Confusion Matrix:
[[21 17]
 [ 3 82]]

Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.55      0.68        38
           1       0.83      0.96      0.89        85

    accuracy                           0.84       123
   macro avg       0.85      0.76      0.78       123
weighted avg       0.84      0.84      0.83       123
```

14 STRATEGY 2: KNN with Distance and Class Weights

```
STRATEGY 3: KNN with ADASYN

After ADASYN - class distribution:
Loan_Status
1    337
0    324
Name: count, dtype: int64

Best parameters: {'metric': 'manhattan', 'n_neighbors': 3}

KNN with ADASYN Results:
Accuracy: 0.7236
ROC-AUC: 0.7200

Confusion Matrix:
[[27 11]
 [23 62]]

Classification Report:
              precision    recall  f1-score   support

           0       0.54      0.71      0.61        38
           1       0.85      0.73      0.78        85

    accuracy                           0.72       123
   macro avg       0.69      0.72      0.70       123
weighted avg       0.75      0.72      0.73       123
```

16 STRATEGY 3: KNN with ADASYN

```
STRATEGY 4: KNN with SMOTE-Tomek

After SMOTE-Tomek - class distribution:
Loan_Status
1    317
0    317
Name: count, dtype: int64

Best parameters: {'metric': 'manhattan', 'n_neighbors': 3}

KNN with SMOTE-Tomek Results:
Accuracy: 0.7480
ROC-AUC: 0.7303

Confusion Matrix:
[[26 12]
 [19 66]]

Classification Report:
              precision    recall  f1-score   support

           0       0.58      0.68      0.63        38
           1       0.85      0.78      0.81        85

    accuracy                           0.75       123
   macro avg       0.71      0.73      0.72       123
weighted avg       0.76      0.75      0.75       123
```

17 STRATEGY 4: KNN with SMOTE-Tomek

```
STRATEGY 5: KNN with Threshold Tuning

Testing different thresholds:
Threshold: 0.25 | Recall 0: 0.5263 | Recall 1: 0.9412 | Acc: 0.8130
Threshold: 0.30 | Recall 0: 0.5263 | Recall 1: 0.9412 | Acc: 0.8130
Threshold: 0.35 | Recall 0: 0.6842 | Recall 1: 0.7765 | Acc: 0.7480
Threshold: 0.40 | Recall 0: 0.6842 | Recall 1: 0.7765 | Acc: 0.7480
Threshold: 0.45 | Recall 0: 0.6842 | Recall 1: 0.7765 | Acc: 0.7480
Threshold: 0.50 | Recall 0: 0.6842 | Recall 1: 0.7765 | Acc: 0.7480
Threshold: 0.55 | Recall 0: 0.6842 | Recall 1: 0.7765 | Acc: 0.7480
Threshold: 0.60 | Recall 0: 0.6842 | Recall 1: 0.7765 | Acc: 0.7480
Threshold: 0.65 | Recall 0: 0.6842 | Recall 1: 0.7765 | Acc: 0.7480
Threshold: 0.70 | Recall 0: 0.7368 | Recall 1: 0.5294 | Acc: 0.5935

Best threshold for Class 0 recall: 0.70

KNN with Optimized Threshold Results:
Accuracy: 0.5935
ROC-AUC: 0.6331

Confusion Matrix:
[[28 10]
 [40 45]]

Classification Report:
...
    accuracy                           0.59       123
   macro avg       0.61      0.63      0.59       123
weighted avg       0.69      0.59      0.61       123
```

19 STRATEGY 5: KNN with Threshold Tuning

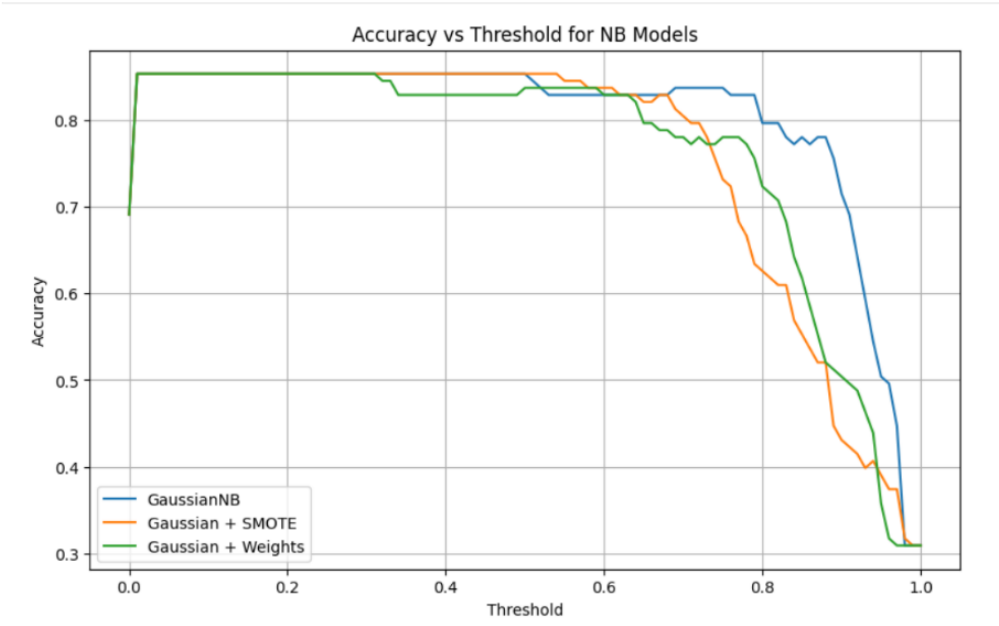
```
STRATEGY 6: KNN with Smaller K + SMOTE

Best parameters: {'metric': 'manhattan', 'n_neighbors': 3, 'weights': 'distance'}

KNN with Smaller K + SMOTE Results:
Accuracy: 0.7480
ROC-AUC: 0.7303

Confusion Matrix:
[[26 12]
 [19 66]]

Classification Report:
              precision    recall  f1-score   support

           0       0.58      0.68      0.63        38
           1       0.85      0.78      0.81        85

    accuracy                           0.75       123
   macro avg       0.71      0.73      0.72       123
weighted avg       0.76      0.75      0.75       123
```

18 STRATEGY 6: KNN with Smaller K + SMOTE

10.3 Naïve Bayes Models

Tested:

- Gaussian NB
- Multinomial NB
- Bernoulli NB



20 Naïve Bayes Threshold Accuracy Curve

NB produced lower performance but offered baseline comparison.

```
Gaussian NB
              precision    recall  f1-score   support

           0       0.95      0.55      0.70        38
           1       0.83      0.99      0.90        85

    accuracy                           0.85       123
   macro avg       0.89      0.77      0.80       123
weighted avg       0.87      0.85      0.84       123
```

22 Gaussian NB

```
Multinomial NB
              precision    recall  f1-score   support

           0       1.00      0.03      0.05        38
           1       0.70      1.00      0.82        85

    accuracy                           0.70       123
   macro avg       0.85      0.51      0.44       123
weighted avg       0.79      0.70      0.58       123
```

21 Multinomial NB

```
Bernoulli NB
              precision    recall  f1-score   support

           0       0.95      0.55      0.70        38
           1       0.83      0.99      0.90        85

    accuracy                           0.85       123
   macro avg       0.89      0.77      0.80       123
weighted avg       0.87      0.85      0.84       123
```

23 Bernoulli NB

```
Gaussian NB + SMOTE
              precision    recall  f1-score   support

           0       0.95      0.55      0.70        38
           1       0.83      0.99      0.90        85

    accuracy                           0.85       123
   macro avg       0.89      0.77      0.80       123
weighted avg       0.87      0.85      0.84       123
```

24 Gaussian NB + SMOTE

```
Gaussian NB + Class Weights
              precision    recall  f1-score   support

           0       0.85      0.58      0.69        38
           1       0.84      0.95      0.89        85

    accuracy                           0.84       123
   macro avg       0.84      0.77      0.79       123
weighted avg       0.84      0.84      0.83       123
```

25 Gaussian NB + Class Weights

10.4 XGBoost Classifier

Strategies tested:

- SMOTE
- Class weights using scale_pos_weight
- ADASYN
- SMOTE-Tomek
- Decision threshold tuning

```
Accuracy: 0.8536585365853658
[[21 17]
 [ 1 84]]
              precision    recall  f1-score   support

           0       0.95      0.55      0.70        38
           1       0.83      0.99      0.90        85

    accuracy                           0.85       123
   macro avg       0.89      0.77      0.80       123
weighted avg       0.87      0.85      0.84       123
```

26 XGBoost Pure Model

```
STRATEGY 1: SMOTE - Synthetic Minority Oversampling

After SMOTE - class distribution:
Loan_Status
1    337
0    337
Name: count, dtype: int64

XGBoost with SMOTE Results:
Accuracy: 0.7886
ROC-AUC: 0.7307

Confusion Matrix:
[[22 16]
 [10 75]]

Classification Report:
              precision    recall  f1-score   support

           0       0.69      0.58      0.63        38
           1       0.82      0.88      0.85        85

    accuracy                           0.79       123
   macro avg       0.76      0.73      0.74       123
weighted avg       0.78      0.79      0.78       123
```

28 STRATEGY 1: SMOTE - Synthetic Minority Oversampling

```
STRATEGY 2: Using Class Weights

Calculated class weights: {0: np.float64(1.594155844155844),
                           1: np.float64(0.728486646884273)}
XGBoost with Class Weights Results:
Accuracy: 0.7724
ROC-AUC: 0.7262

Confusion Matrix:
[[23 15]
 [13 72]]

Classification Report:
              precision    recall  f1-score   support

           0       0.64      0.61      0.62        38
           1       0.83      0.85      0.84        85

    accuracy                           0.77       123
   macro avg       0.73      0.73      0.73       123
weighted avg       0.77      0.77      0.77       123
```

27 STRATEGY 2: Using Class Weights

```
eshold Tuning

Testing different thresholds:
Threshold: 0.30 | Recall Class 0: 0.5526 | Recall Class 1: 0.9882
Threshold: 0.35 | Recall Class 0: 0.5526 | Recall Class 1: 0.9882
Threshold: 0.40 | Recall Class 0: 0.5526 | Recall Class 1: 0.9882
Threshold: 0.45 | Recall Class 0: 0.5526 | Recall Class 1: 0.9647
Threshold: 0.50 | Recall Class 0: 0.5789 | Recall Class 1: 0.8824
Threshold: 0.55 | Recall Class 0: 0.5789 | Recall Class 1: 0.8471
Threshold: 0.60 | Recall Class 0: 0.7105 | Recall Class 1: 0.7412
Threshold: 0.65 | Recall Class 0: 0.7632 | Recall Class 1: 0.6353

Best threshold: 0.65

XGBoost with Optimized Threshold Results:
Accuracy: 0.6748
ROC-AUC: 0.6992

Confusion Matrix:
[[29  9]
 [31 54]]

Classification Report:
              precision    recall  f1-score   support
...
    accuracy                           0.67       123
   macro avg       0.67      0.70      0.66       123
weighted avg       0.74      0.67      0.69       123
```

29 STRATEGY 3: Threshold Tuning

```
STRATEGY 4: ADASYN - Adaptive Synthetic Sampling

After ADASYN - class distribution:
Loan_Status
1    337
0    323
Name: count, dtype: int64

XGBoost with ADASYN Results:
Accuracy: 0.8049
ROC-AUC: 0.7351

Confusion Matrix:
[[21 17]
 [ 7 78]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.55      0.64        38
           1       0.82      0.92      0.87        85

    accuracy                           0.80       123
   macro avg       0.79      0.74      0.75       123
weighted avg       0.80      0.80      0.80       123
```

31 STRATEGY 4: ADASYN - Adaptive Synthetic Sampling

```
STRATEGY 5: SMOTE + Tomek Links (Hybrid Approach)

After SMOTE-Tomek - class distribution:
Loan_Status
1    319
0    319
Name: count, dtype: int64

XGBoost with SMOTE-Tomek Results:
Accuracy: 0.7805
ROC-AUC: 0.7320

Confusion Matrix:
[[23 15]
 [12 73]]

Classification Report:
              precision    recall  f1-score   support

           0       0.66      0.61      0.63        38
           1       0.83      0.86      0.84        85

    accuracy                           0.78       123
   macro avg       0.74      0.73      0.74       123
weighted avg       0.78      0.78      0.78       123
```

30 STRATEGY 5: SMOTE + Tomek Links (Hybrid Approach)

11. Model Comparison & Final Selection

Model	Best Strategy	Pros	Cons
Decision Tree	Balanced Class Weights	Best interpretability; balanced performance; transparent decision logic	Slight risk of overfitting
KNN	Scaling + SMOTE	Simple; intuitive; performs better after scaling	Sensitive to noise; slow with large data
Naïve Bayes	SMOTE / Class Weights	Very fast; computationally efficient	Lower accuracy; strong assumptions
XGBoost	SMOTE / Class Weights	High performance; handles complex patterns well	Less interpretable; more complex

- **Final Model: Decision Tree (Balanced Class Weights)**

12. New Customer Prediction Pipeline

Pipeline components loaded from joblib:

- imputers for missing values
- dependents mode dictionary
- marriage rule
- RF model for LoanAmount
- log caps
- One-Hot Encoder
- scaler
- Decision Tree final model
- exact training column order

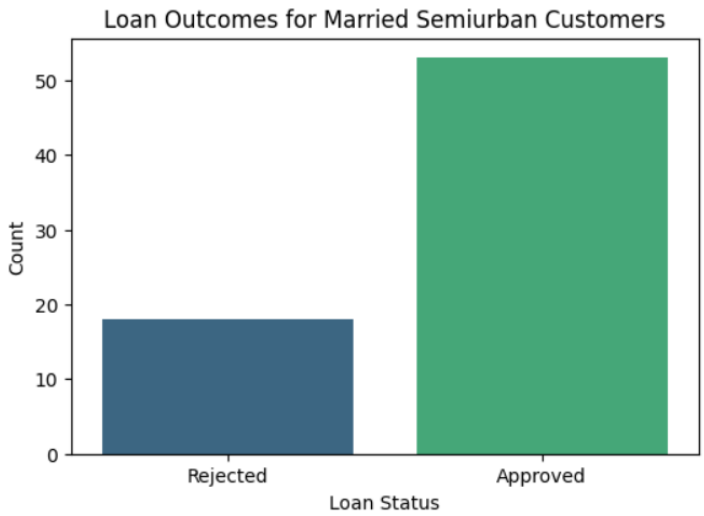
	loan id	Loan_Status_Pred
0	0	1
1	1	1
2	2	1
3	3	1
4	4	1

32 New Customer Final Prediction Table

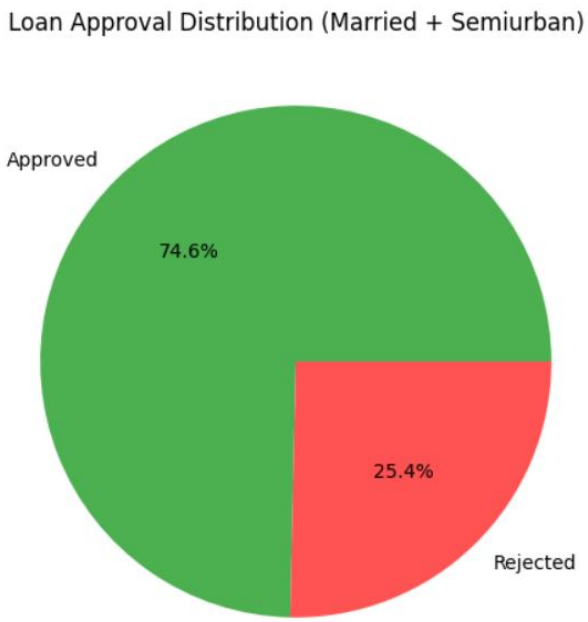
13. Business Validation: Married + Semiurban Analysis

Historical dataset: **80% approval rate**
Model predictions: **~75% approval rate**

This proves the model generalizes well and maintains realistic behavior.



33 Loan Outcomes for Married Semiurban Customers



34 Loan Approval Distribution (Married + Semiurban)

14. Conclusion

The project achieved a full end-to-end solution:

- Clean preprocessing pipeline
- Outlier handling with zero data leakage
- Class imbalance solutions tailored to each classifier
- Full set of model experiments
- Deployment-ready new customer pipeline
- Real-world behavioral validation

This project demonstrates strong technical and practical foundations in machine learning for financial applications.

15. Appendix — GitHub Repository

The full project code, preprocessing steps, and saved models are available at:

https://github.com/OmarAbouraia/Loan_Prediction_Project.git